FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Mark Anthony G. Petrimetre
Section:	AN-41
Professor:	Crisola Tan
Date:	10/1/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

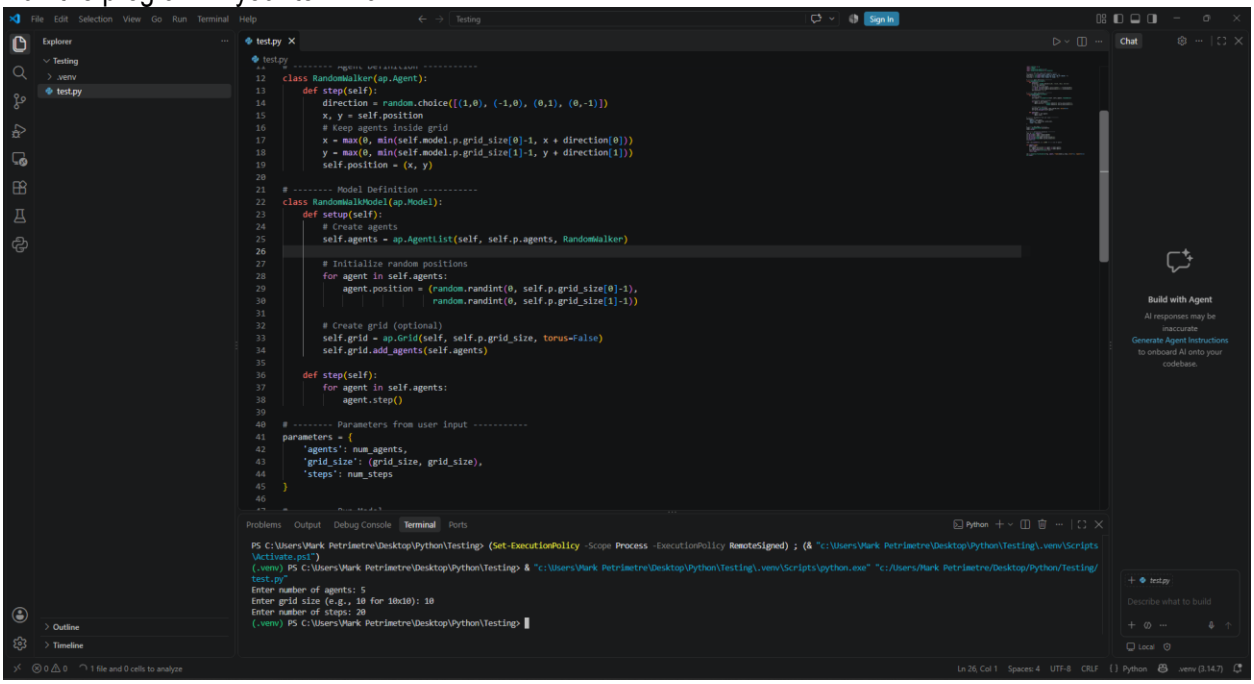
# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

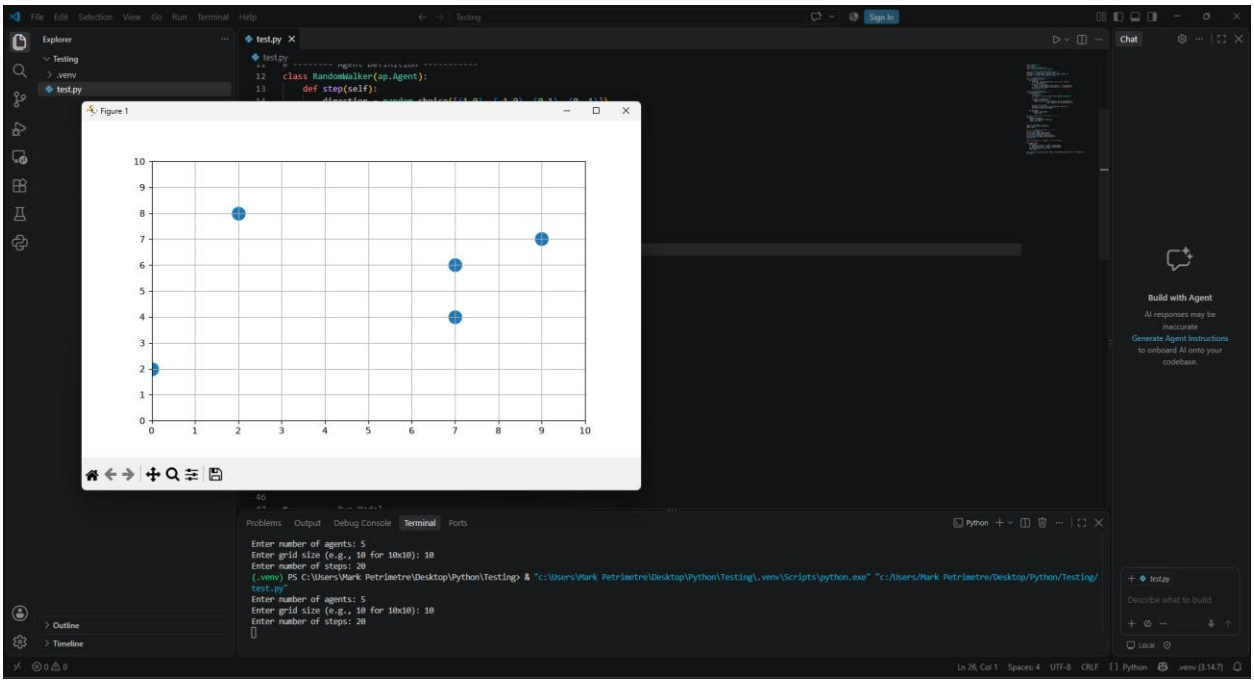
def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.



3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)



These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

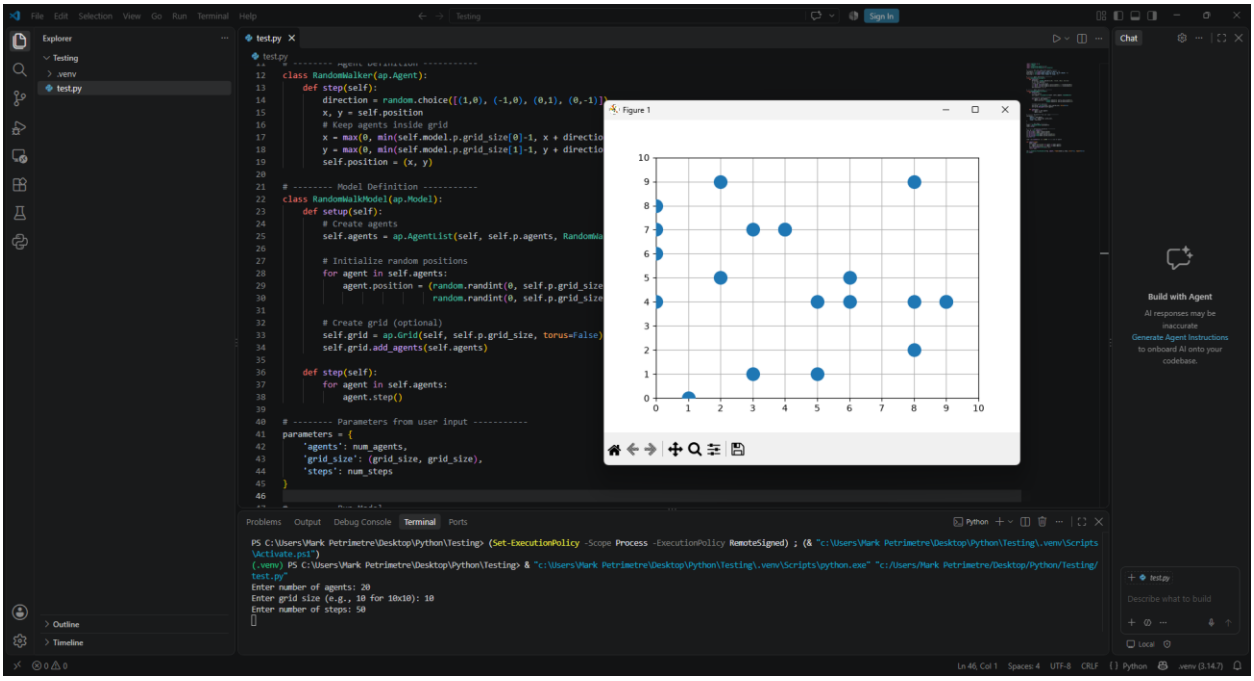
1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.



Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩
Code:

```
import agentpy as ap
import random
import matplotlib.pyplot as plt

class RandomWalker(ap.Agent):
    def setup(self):
        # Pick an initial position within the grid boundaries
        gx, gy = self.model.p.grid_size
        self.position = (random.randint(0, gx - 1), random.randint(0, gy - 1))

        # Track path history throughout the run
        self.path = [self.position]

    def step(self):
```

```
        gx, gy = self.model.p.grid_size
        x, y = self.position

        # Candidate directions: Right, Left, Up, Down
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        # Task 2: Movement rules (Unbiased vs Biased Direction)
        if self.model.p.movement_mode == 'drift_right':
            # 55% chance to move right, remaining probability split evenly
            weights = [0.55, 0.15, 0.15, 0.15]
            dx, dy = random.choices(directions, weights=weights)[0]
        else:
            # Standard uniform random walk
            dx, dy = random.choice(directions)

        # Boundary clamping to stay inside the grid
        new_x = max(0, min(gx - 1, x + dx))
        new_y = max(0, min(gy - 1, y + dy))
        next_pos = (new_x, new_y)

        # Collision avoidance check against other agents
        if self.model.p.avoid_collision:
            occupied = {other.position for other in self.model.agents if other != self}
            if next_pos in occupied:
                # If target cell is already occupied, stay in current position
                next_pos = (x, y)

        self.position = next_pos
        self.path.append(self.position)


class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        self.agents.step()

    def end(self):
        # Log final coordinates and quadrant distribution
        print(f"\n--- Final Position Analysis: {self.p.scenario_name} ---")
        mid_x = self.p.grid_size[0] / 2
```

```
        mid_y = self.p.grid_size[1] / 2

        q_counts = {"Q1 (Top-Right)": 0, "Q2 (Top-Left)": 0, "Q3 (Bottom-Left)": 0, "Q4 (Bottom-Right)": 0}

        for idx, agent in enumerate(self.agents):
            x, y = agent.position
            print(f"Agent {idx + 1} final position: ({x}, {y})")

            if x >= mid_x and y >= mid_y:
                q_counts["Q1 (Top-Right)"] += 1
            elif x < mid_x and y >= mid_y:
                q_counts["Q2 (Top-Left)"] += 1
            elif x < mid_x and y < mid_y:
                q_counts["Q3 (Bottom-Left)"] += 1
            else:
                q_counts["Q4 (Bottom-Right)"] += 1

        print("Distribution across quadrants:")
        for q, count in q_counts.items():
            print(f"  {q}: {count}")


if __name__ == "__main__":
    # Console prompts
    num_agents = int(input("Enter number of agents: "))
    grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
    num_steps = int(input("Enter number of steps: "))

    # Simulation parameters
    parameters = {
        'scenario_name': 'Standard vs Collision-Avoidance Walk',
        'agents': num_agents,
        'grid_size': (grid_size, grid_size),
        'steps': num_steps,
        'movement_mode': 'standard',     # Options: 'standard' or 'drift_right'
        'avoid_collision': True          # Toggle collision avoidance
    }

    model = RandomWalkModel(parameters)
    model.run()

    # Plotting agent trajectories on the grid
    fig, ax = plt.subplots(figsize=(7, 7))
    ax.set_xlim(-0.5, grid_size - 0.5)
```

```
    ax.set_ylim(-0.5, grid_size - 0.5)
    ax.set_xticks(range(grid_size))
    ax.set_yticks(range(grid_size))
    ax.grid(True, linestyle="--", alpha=0.6)
    ax.set_title("Agent Paths and Final Positions")

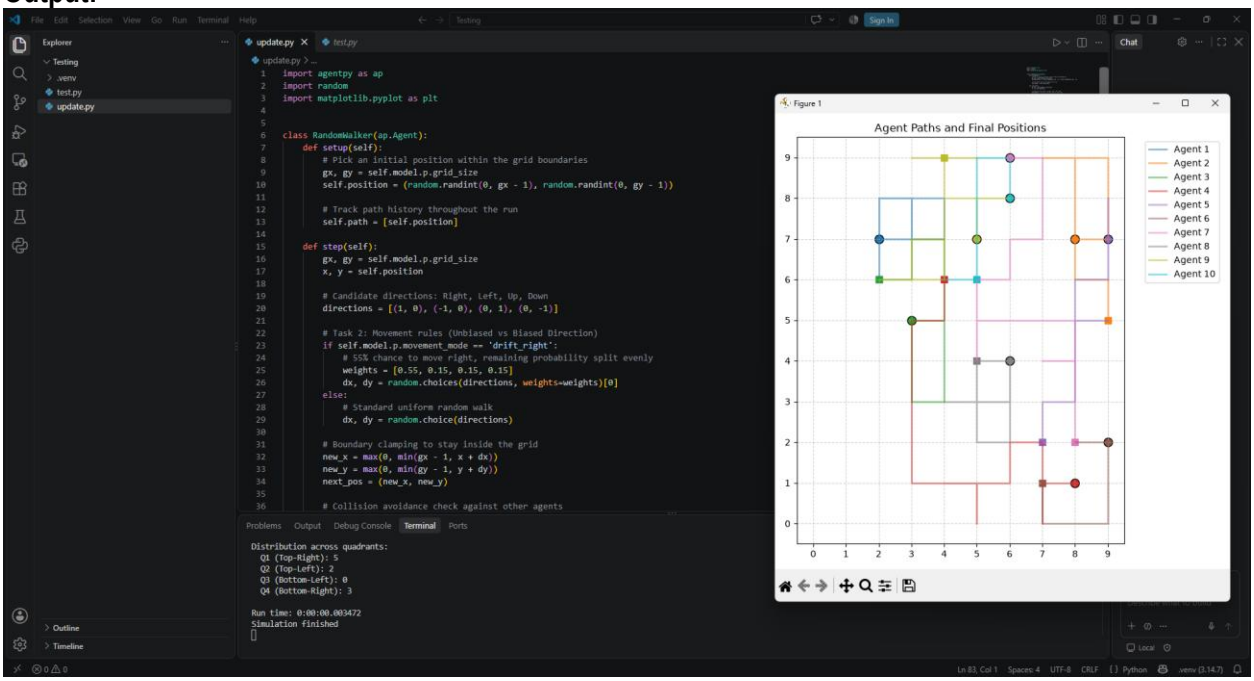
    colors = plt.cm.tab10.colors

    for i, agent in enumerate(model.agents):
        c = colors[i % len(colors)]
        xs = [pos[0] for pos in agent.path]
        ys = [pos[1] for pos in agent.path]

        # Draw trajectory line
        ax.plot(xs, ys, color=c, alpha=0.5, linewidth=2, label=f"Agent {i + 1}")
        # Mark start (square) and end (circle)
        ax.scatter(xs[0], ys[0], color=c, marker='s', s=50)
        ax.scatter(xs[-1], ys[-1], color=c, marker='o', s=90, edgecolors='black')

    ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.show()
```

Output:



Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
    - The grid gets super crowded really fast. Without collision handling, agents just keep stacking on the exact same tiles. But if collision avoidance is turned on, they end up blocking each other in almost every direction, causing gridlock where barely anyone can move.
2. How does increasing the number of steps affect the **spread of agents**?
    - It gives them more time to scatter across the board. At the start, they might be near wherever they spawned, but as more steps pass, they spread out more evenly until they're basically dispersed all over the available grid space.
3. Do the agents ever move outside the grid? Why or why not?
    - No, they don't. In the code, the coordinates are clamped using max(0, min(grid_size - 1, ...)) whenever they take a step. So if a random move tries to push an agent past the edges or into negative values, it just gets capped back to the boundary.
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
    - You can use it to model **epidemic transmission (disease spread)**. Agents can represent individuals in a community moving randomly between locations, and if an "infected" agent lands on or near a healthy agent's tile, there's a probability that the infection spreads.

Answer the Questions ⇩